

Python Exercise: Knapsack, QUBO, and Quantum-Inspired Optimization

Master-Level Python Course

1 Introduction

The exercise introduces:

- Python programming for optimization,
- the classical *knapsack problem*,
- the idea of a *QUBO* formulation,
- and two software environments for simulating quantum approaches.

Even students who are already strong in Python may find the modeling concepts new, especially if they have not studied mixed-integer linear programming or QUBO formulations before.

2 The Knapsack Problem

The knapsack problem is a classical optimization problem. We are given a set of items. Each item has:

- a **value** (or profit),
- a **weight**,
- and we must decide whether to include it in the knapsack.

The objective is to maximize the total value of the selected items, subject to a capacity limit on the total weight.

A standard mathematical formulation is:

$$\max \sum_{i \in P} c_i x_i$$

subject to

$$\sum_{i \in P} a_i x_i \leq b$$

$$x_i \in \{0, 1\} \quad \forall i \in P$$

where:

- P is the set of items,
- c_i is the value of item i ,
- a_i is the weight of item i ,
- b is the capacity of the knapsack,
- $x_i = 1$ if item i is selected, and $x_i = 0$ otherwise.

How the instance is described in the JSON file

Each attached JSON file stores one knapsack instance using the following structure:

```
{
  "sets": {
    "P": ["1", "2", "3", "4"]
  },
  "parameters": {
    "b": 10,
    "C": {
      "1": 8,
      "2": 6,
      "3": 5,
      "4": 4
    },
    "a": {
      "1": 6,
      "2": 4,
      "3": 3,
      "4": 2
    }
  }
}
```

The meaning of the fields is:

- `sets["P"]`: the list of item identifiers,
- `parameters["b"]`: the knapsack capacity,
- `parameters["C"]`: the value of each item,
- `parameters["a"]`: the weight of each item.

For the example above:

- the items are 1, 2, 3, 4,
- the capacity is 10,
- the values are 8, 6, 5, 4,
- the weights are 6, 4, 3, 2.

then the knapsack instance is:

$$\begin{aligned} \max \quad & 8x_1 + 6x_2 + 5x_3 + 4x_4 \\ \text{s.t.} \quad & 6x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{aligned}$$

When reading the file in Python, students should reconstruct the ordered lists of values and weights by following the order of the items in P.

3 The QUBO Formulation of the Knapsack Problem

QUBO stands for **Quadratic Unconstrained Binary Optimization**. A QUBO problem has the general form

$$\min x^\top Qx$$

where x is a vector of binary variables and Q is a matrix of coefficients.

The main idea is that a constrained optimization problem can be transformed into an unconstrained one by adding a penalty term to the objective function.

For the knapsack problem, one possible approach is to penalize violations of the capacity constraint. Conceptually, we want to maximize the value of the selected items while discouraging infeasible solutions.

A penalized form can be written as

$$\max \left(\sum_{i \in P} c_i x_i - \lambda \left(\sum_{i \in P} a_i x_i - b \right)^2 \right)$$

where $\lambda > 0$ is a penalty parameter.

In practice, a more accurate QUBO construction often introduces additional binary variables, for example to encode a slack variable. This allows us to represent the capacity constraint more precisely while maintaining a quadratic binary formulation.

The QUBO reformulation is important because many quantum optimization methods and quantum-inspired algorithms are designed to work with binary quadratic models rather than with standard constrained linear models.

4 Programming Tasks

Students must write four Python programs. Each program must read the knapsack instance from one of the provided JSON files and solve the problem in a different way.

Task 1: Classical solution with Pyomo and Gurobi

Write a Python program that:

- reads the JSON file,
- extracts the items, values, weights, and capacity,
- formulates the standard knapsack problem as a binary optimization model,
- solves it using **Pyomo** and **Gurobi**,
- prints the selected items, total value, and total weight.

Brief explanation of the libraries.

- **Pyomo** is a Python library for mathematical optimization. It allows the user to define variables, objective functions, and constraints using a clear algebraic syntax.
- **Gurobi** is a high-performance optimization solver. It can solve linear, integer, and quadratic optimization problems efficiently.

Task 2: QUBO-style solution with Pyomo and Gurobi

Write a Python program that:

- reads the same JSON file,
- converts the knapsack problem into a QUBO-style binary quadratic model,
- solves the resulting quadratic formulation using `Pyomo` and `Gurobi`,
- compares the result with the standard formulation.

Students should pay attention to:

- the role of the penalty coefficient,
- the possible use of extra binary variables,
- the difference between the original constrained formulation and the QUBO reformulation.

Task 3: Quantum simulation with Qiskit

Write a Python program that:

- reads the JSON file,
- builds the knapsack problem,
- converts it into a QUBO formulation,
- solves it using a quantum computing simulator in `Qiskit`,
- prints the selected items and the corresponding objective value.

Brief explanation of the library. `Qiskit` is a Python ecosystem for quantum computing. It provides tools for building quantum circuits, running them on simulators, and interfacing with quantum hardware. In optimization, `Qiskit` can be used to represent combinatorial problems and solve them with variational quantum algorithms such as QAOA. The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical algorithm designed to find approximate solutions to combinatorial optimization problems (such as knapsack) on short-term quantum devices.

Task 4: Quantum-inspired solution with dimod

Write a Python program that:

- reads the JSON file,
- formulates the knapsack problem as a QUBO or binary quadratic model,
- solves it using `dimod`,
- prints the selected items and the corresponding objective value.

Brief explanation of the library. `dimod` is a Python library from the D-Wave Ocean ecosystem for working with binary quadratic models. It provides data structures and solvers for QUBO and Ising problems, and it can be used with exact solvers, simulated annealing, and quantum annealing workflows.

5 Repeated Runs and Interpretation of the Results

Students must perform repeated runs with the two quantum software environments:

- Qiskit,
- dimod.

They must then explain why different solutions, and sometimes even suboptimal solutions, may be obtained.

The explanation should include the following points:

- quantum and quantum-inspired methods are often **heuristic** rather than exact,
- some algorithms depend on random initialization or stochastic search,
- the number of repetitions or reads may affect the result,
- the penalty coefficient in the QUBO formulation may strongly influence solution quality,
- simulators imitate the behavior of quantum or annealing-based approaches, but they do not guarantee that the global optimum is always found,
- small changes in parameters may produce different outcomes.

Students should compare the results of the quantum approaches with the exact classical solution obtained with Pyomo and Gurobi.

6 Suggested Deliverables

Each student or group should submit:

- the four Python programs,
- a short report describing the modeling choices,
- a comparison of the solutions obtained,
- a discussion of the repeated runs of the quantum approaches,
- a brief reflection on the difference between exact optimization and heuristic or quantum-inspired optimization.

7 Optional Extensions for Advanced Students

Students with stronger programming backgrounds may extend the exercise by exploring one or more of the following:

- testing larger knapsack instances,
- generating random instances automatically,
- tuning the penalty parameter in the QUBO model,
- comparing runtime and solution quality,
- visualizing the selected solutions,
- analyzing when the QUBO formulation becomes harder to solve.